

## ASSIGNMENT 7.3

2303A52488

### TASK 1: FIXING SYNTAX ERRORS

ORIGINAL BUGGY CODE (MISSING COLON):

```
DEF ADD(A, B) RETURN A + B
```

AI PROMPT USED: "FIX THIS PYTHON SYNTAX ERROR IN THE FUNCTION

DEFINITION." TERMINAL EXECUTION OF BUGGY CODE:

```
$ PYTHON3 BUGGY_SYNTAX.PY FILE "BUGGY_SYNTAX.PY", LINE 1 DEF
ADD(A, B)      ^ SYNTAXERROR: INVALID SYNTAX
```

AI CORRECTED CODE:

```
DEF ADD(A, B): RETURN A + B
```

TERMINAL EXECUTION OF CORRECTED CODE:

```
$ PYTHON3 FIXED_SYNTAX.PY
5
```

AI EXPLANATION: THE COLON AFTER THE FUNCTION PARAMETERS IS MISSING, WHICH IS REQUIRED FOR PYTHON FUNCTION DEFINITIONS TO INDICATE THE START OF THE FUNCTION BODY. ADDED THE COLON TO FIX THE SYNTAXERROR.

### TASK 2: DEBUGGING LOGIC ERRORS IN LOOPS

ORIGINAL BUGGY CODE (INFINITE LOOP):

```
COUNT = 10 WHILE
COUNT > 0:
PRINT(COUNT)  #
COUNT -= 1  #
FORGOTTEN
DECREMENT
```

AI PROMPT USED: "DEBUG THIS INFINITE LOOP IN PYTHON."

TERMINAL EXECUTION (HANGS, CTRL+C TO STOP):

```
$ PYTHON3 BUGGY_LOOP.PY
10
10
10 ^CTRACEBACK (MOST RECENT CALL
LAST):  FILE "BUGGY_LOOP.PY", LINE 3,
IN <MODULE>  PRINT(COUNT)
KEYBOARDINTERRUPT
```

AI CORRECTED CODE:

```
COUNT = 10 WHILE COUNT > 0:  PRINT(COUNT)  COUNT -= 1
```

TERMINAL EXECUTION OF CORRECTED CODE:

```
$ PYTHON3 FIXED_LOOP.PY
10
9
8
7
6
5
4
3
2
```

1

AI EXPLANATION: THE LOOP LACKS A DECREMENT ON 'COUNT', CAUSING INFINITE ITERATION SINCE THE CONDITION NEVER BECOMES FALSE. ADDED 'COUNT -= 1' INSIDE THE LOOP TO FIX THE LOGIC ERROR.

### TASK 3: HANDLING RUNTIME ERRORS - DIVISION BY ZERO

ORIGINAL BUGGY CODE:

```
DEF DIVIDE(X, Y):  
    RETURN X / Y  
PRINT(DIVIDE(10, 0))
```

AI PROMPT USED: "FIX DIVISION BY ZERO RUNTIME ERROR IN THIS

FUNCTION." TERMINAL EXECUTION OF BUGGY CODE:

```
$ PYTHON3 BUGGY_DIVIDE.PY TRACEBACK (MOST RECENT CALL LAST): FILE  
"BUGGY_DIVIDE.PY", LINE 4, IN <MODULE>    PRINT(DIVIDE(10, 0))  
~~~~~ FILE "BUGGY_DIVIDE.PY", LINE 2, IN DIVIDE    RETURN X / Y  
~~^~~ ZERODIVISIONERROR: DIVISION BY ZERO
```

AI CORRECTED CODE:

```
DEF DIVIDE(X, Y):    TRY:  
    RETURN X / Y    EXCEPT  
ZERODIVISIONERROR:  
    RETURN "CANNOT DIVIDE BY ZERO"  
PRINT(DIVIDE(10, 0))
```

TERMINAL EXECUTION OF CORRECTED CODE:

```
$ PYTHON3 FIXED_DIVIDE.PY  
CANNOT DIVIDE BY ZERO
```

AI EXPLANATION: ADDED TRY-EXCEPT BLOCK TO CATCH ZERO\_DIVISION\_ERROR SPECIFICALLY.  
RETURNS A SAFE MESSAGE INSTEAD OF CRASHING WHEN DENOMINATOR IS ZERO.

#### TASK 4: DEBUGGING CLASS DEFINITION ERRORS

ORIGINAL BUGGY CODE (MISSING SELF):

```
class Student:
    def __init__(name):
        self.name = name
```

AI PROMPT USED: "CORRECT THIS PYTHON CLASS

CONSTRUCTOR ERROR." TERMINAL EXECUTION OF BUGGY

CODE:

```
$ python3 buggy_class.py
Traceback (most recent call last):
  File "buggy_class.py", line 5, in <module>
    s = Student("Alice")
TypeError: Student.__init__() missing 1 required positional argument: 'name'
```

AI CORRECTED CODE:

```
class Student:
    def __init__(self, name):
        self.name = name

s = Student("Alice")
print(s.name)
```

TERMINAL EXECUTION OF CORRECTED CODE:

```
$ python3 fixed_class.py
Alice
```

AI EXPLANATION: THE INIT METHOD REQUIRES 'SELF' AS THE FIRST PARAMETER TO REFER TO THE INSTANCE. ADDED 'SELF' TO FIX THE TYPEERROR.

#### TASK 5: RESOLVING INDEX ERRORS IN LISTS

**ORIGINAL BUGGY CODE:**

```
MY_LIST = [1, 2, 3] PRINT(MY_LIST[^1_5])
```

**AI PROMPT USED: "FIX INDEX OUT OF RANGE ERROR IN THIS LIST**

**ACCESS." TERMINAL EXECUTION OF BUGGY CODE:**

```
$ PYTHON3 BUGGY_INDEX.PY TRACEBACK (MOST RECENT CALL LAST): FILE
"BUGGY_INDEX.PY", LINE 2, IN <MODULE>  PRINT(MY_LIST[^1_5]) INDEXERROR: LIST
INDEX OUT OF RANGE
```

**AI CORRECTED CODE:**

```
MY_LIST = [1, 2, 3] IF
LEN(MY_LIST) > 5:
    PRINT(MY_LIST[^1_5]) ELSE:
    PRINT("INDEX OUT OF RANGE")
```

**TERMINAL EXECUTION OF CORRECTED CODE:**

```
$ PYTHON3 FIXED_INDEX.PY
INDEX OUT OF RANGE
```

**AI EXPLANATION: CHECKED LIST LENGTH BEFORE ACCESSING INDEX TO PREVENT INDEXERROR.  
USED CONDITIONAL BOUNDS CHECKING FOR SAFE ACCESS.**